

SpringMVC相关知识点

一、关于Spring集成SpringMVC的父子容器

DispatcherServlet初始化init方法的时候（1、获得一个SpringMVC的ApplicationContext容器，然后其继承Spring容器，成为父子容器 2、在容器初始化完成后，通过监听事件完成Spring MVC九大组件的注册），会去初始化Spring容器,是不是通过ServletContextListener的方法，然后创建Spring容器，然后将Spring容器放到ServletContext中，然后在其init方法过程中，创建完SpringMVC后，让后将Spring作为父容器，设置给SpringMVC容器

总流程：

- `ServletContextListener` 启动，Spring 容器初始化并注入到 `ServletContext`。
- `DispatcherServlet` 在初始化过程中，会使用这个容器。
- `DispatcherServlet` 会创建自己的 Spring MVC `webApplicationContext`，并将父容器（Spring 容器）作为其父容器。
- 完成 Spring MVC 初始化后，Spring MVC 容器和 Spring 容器之间建立了父子关系。

这种方式确保了 Spring MVC 可以继承和使用 Spring 容器中管理的 Bean，同时也能够单独管理它自己的 Bean 配置。

参考：

Spring MVC 初始化过程

1. `web.xml` 配置或 Spring Boot 自动配置：

- 在传统的Spring MVC应用中，`DispatcherServlet` 是在 `web.xml` 中进行配置的。而在 Spring Boot 中，`DispatcherServlet` 是由 Spring Boot 自动配置的，它不需要显式地在 `web.xml` 中配置。
- 如果是传统的Spring MVC应用，`DispatcherServlet` 会在 `web.xml` 中配置，Spring Boot 会自动完成Servlet的配置。

2. `ServletContextListener` 启动：

- `ServletContextListener` 是一个监听器，它会在 Web 应用启动时被触发。Spring Boot通过 `SpringServletContainerInitializer` 来进行Spring的初始化。
- `SpringServletContainerInitializer` 会调用 `ServletContext.addListener()` 方法，注册一个监听器，监听 `ServletContext` 的生命周期。这时Spring会通过监听器在 `ServletContext` 中创建和初始化Spring容器（`ApplicationContext`）。

3. 初始化 Spring 容器：

- `ServletContextListener` 会创建 Spring 容器。Spring Boot 默认会使用 `AnnotationConfigApplicationContext` 或 `GenericWebApplicationContext` 作为 Spring 容器，并将它注入到 `ServletContext` 中。
- 此时，Spring 容器并没有直接与 Spring MVC 进行绑定，而是先独立运行，负责处理普通的 Spring Bean 管理、依赖注入等。

4. `DispatcherServlet` 初始化：

- `DispatcherServlet` 是 Spring MVC 的前端控制器，它会通过 `init()` 方法来初始化 Spring MVC 的相关组件。`DispatcherServlet` 的 `init()` 方法会在初始化时查找和加载 `WebApplicationContext`。
- `DispatcherServlet` 会创建一个 `WebApplicationContext`，该上下文是 Spring MVC 的专用容器。

5. 将 Spring 容器作为父容器：

- 在 Spring MVC 中，`DispatcherServlet` 需要一个容器来管理它的组件。通常情况下，`WebApplicationContext` 会继承自 `ApplicationContext`，即它是一个包含 Spring 容器的上下文。
- Spring Boot 会将之前创建的父容器（通常是 `ApplicationContext`）作为 `WebApplicationContext` 的父容器。在这种情况下，`DispatcherServlet` 创建的 Spring MVC 容器会作为子容器，它继承了父容器的所有配置和 bean。

6. Spring MVC 初始化完成：

- 在 `DispatcherServlet` 的初始化过程中，它会设置自己的 `ApplicationContext` 为 Spring MVC 的上下文，并且通过父容器来继承和共享父容器的 bean 配置。这使得整个 Spring MVC 的应用上下文可以访问父容器中的所有普通 Spring Bean。

为什么要有父子容器：

- 单一职责原则，划分框架边界，Spring MVC容器负责管理Controller层，Service和dao层通过spring容器去管理
- 规范整体框架：使得父容器无法访问子容器（即service无法访问controller），子容器可以访问父容器（controller依赖service）
- 方便子容器的切换：把web层从spring mvc替换成struts，只需要将spring-mvc.xml替换成struts配置文件struts.xml即可
- 节省重复bean 的创建

二、SpringMVC处理请求

请求处理流程：

1. 请求到达 `DispatcherServlet`。
2. `DispatcherServlet` 通过 `HandlerMapping` 查找对应的处理器（Controller），并封装执行器链。
3. `DispatcherServlet` 使用 `HandlerAdapter` 调用处理器方法，并调用执行器链的各个方法。
4. 执行前，`HandlerInterceptor` 可拦截处理器方法进行操作。
5. 处理器方法执行并返回 `ModelAndView`。
6. `ViewResolver` 解析视图并渲染。
7. 渲染完成后，将响应返回给客户端。

1. 请求到达前端控制器 (DispatcherServlet)

所有的 HTTP 请求都会首先到达 Spring MVC 的前端控制器 `DispatcherServlet`。这是 Spring MVC 请求处理流程的入口。

2. DispatcherServlet 通过 HandlerMapping 查找处理器

`DispatcherServlet` 会通过 `HandlerMapping` 机制来查找合适的 **处理器 (Controller)**，也就是确定哪个方法应该处理这个请求。`HandlerMapping` 会根据请求的 URL 映射到一个对应的控制器方法（例如 `@RequestMapping` 注解的处理方法）。

3. HandlerAdapter 调用处理器

一旦找到处理器，`DispatcherServlet` 会通过 `HandlerAdapter` 来调用对应的处理器方法。

`HandlerAdapter` 是用来将请求传递给相应的处理器并调用其方法的组件。Spring MVC 允许有多种不同的 `HandlerAdapter` 实现，来适配不同的请求处理方式（例如，注解驱动的控制​​器、传统的 `Controller` 接口实现等）。

4. 处理器方法执行

`HandlerAdapter` 找到对应的处理器后，会调用该处理器的方法来处理请求。在这个阶段，控制器方法会执行其逻辑（如读取请求参数、执行业务逻辑、返回视图名等）。

5. 方法前后 (HandlerInterceptor)

在请求的处理过程中，Spring MVC 还可以通过拦截器 (`HandlerInterceptor`) 来进行方法前、后操作。例如，日志记录、权限验证、参数处理等。

- `preHandle()`：方法执行前的处理。
- `postHandle()`：方法执行后的处理。
- `afterCompletion()`：请求处理完成后（视图渲染之后）的清理工作。

拦截器会在处理器方法执行前后进行拦截，可以执行一些额外的操作。

6. 返回 ModelAndView

处理器方法执行完成后，返回一个 `ModelAndView` 对象，它包含了模型数据和视图信息。此时，Spring MVC 处理器方法已经完成了请求处理，视图解析和渲染的工作交给了后续的组件。

7. 视图解析器 (ViewResolver)

`DispatcherServlet` 会根据 `ModelAndView` 中的视图信息，使用 `ViewResolver` 来解析最终的视图（如 JSP、Thymeleaf 等），并将模型数据传递给视图进行渲染。

8. 渲染视图并响应客户端

一旦视图解析完成，视图会被渲染（比如渲染成 HTML 页面），然后将响应返回给客户端。

概括：

请求到达前端控制器，然后通过处理器映射器找到对应的controller处理器对象，以及对应的执行器链，然后通过处理器适配器对执行链的方法，并且调用执行器链的执行前、执行后、请求处理完成后的方法，并调用处理器的方法，最后就是返回。。。

三、异常处理器

是通过Spring MVC来实现的，在九大组件注册的时候，会将异常处理组件也注册进去。

工作原理：

- Spring MVC 的 `@ControllerAdvice` 和 `@ExceptionHandler` 是基于 `HandlerExceptionResolver` 机制来实现的。
- 当请求处理过程中抛出异常时，Spring MVC 会通过 `HandlerExceptionResolver` 查找对应的异常处理方法。
- 如果该异常被 `@ControllerAdvice` 中的 `@ExceptionHandler` 方法捕获处理，Spring 会调用这个方法并返回相应的结果。